

FIT – HPUni BECOME A BETTER DEVELOPER

LAB 1.1 : KIỂU DỮ LIỆU, TOÁN TỬ, IF ELSE

1. KIỂU DỮ LIỆU, CẤU TRÚC Rẽ NHÁNH

Một số lưu ý về kiểu dữ liệu char:

Mỗi kí tự các bạn hay gặp đều là các ký tự trong bảng mã ASCII. Bảng mã này có 256 giá trị từ 0 - 255. Mỗi kí tự sẽ được gán một mã ASCII. Các mã ASCII các bạn cần nắm được:

- Các kí tự từ 'a' - 'z' có mã ASCII từ 97 - 122.
- Các kí tự từ 'A' - 'Z' có mã ASCII từ 65 - 90.
- Các kí tự từ '0' - '9' có mã ASCII từ 48 - 57.

Vì vậy để kiểm tra các kí tự là chữ cái, in hoa, in thường hay chữ số các bạn sử dụng dải mã ASCII tương ứng.

Chú ý: Kiểu dữ liệu char là kí tự nhưng bạn có thể sử dụng nó như một số, chính là mã ASCII đại diện cho nó để cộng, trừ, nhân, chia như là với số nguyên thông thường.

2. DANH SÁCH BÀI TẬP

Bài 1. Phần nguyên, phần dư	3
Bài 2. Tính toán giá trị biểu thức.....	3
Bài 3. Tính toán giá trị biểu thức 2.....	4
Bài 4. Tính tổng, hiệu, tích, thương.....	5
Bài 5. Tính chu vi, diện tích hình tròn.....	6
Bài 6. Tính khoảng cách.....	6
Bài 7. Chuyển đơn vị đo C và F	7
Bài 8. Tính tổng 1.....	8
Bài 9. Tính tổng 2.....	8
Bài 10. Tính tổng 3.....	9
Bài 11. Tính tổng 4.....	9
Bài 12. Tính tổng 5.....	10
Bài 13. Số chia hết lớn nhất.....	11
Bài 14. Số chia hết nhỏ nhất.....	11
Bài 15. Kiểm tra số chia hết cho 3 và 5.....	12
Bài 16. Kiểm tra năm nhuận.....	12
Bài 17. In ra số ngày của tháng.....	13

Bài 18. Kiểm tra chữ in thường	15
Bài 19. Kiểm tra in hoa	15
Bài 20. Kiểm tra chữ cái.....	16
Bài 21. Kiểm tra chữ số	17
Bài 22. Chuyển ký tự hoa thành thường	17
Bài 23. Chuyển ký tự thường thành ký tự hoa.....	18
Bài 24. Chữ cái kế tiếp.....	18
Bài 25. Tam giác hợp lệ	19
Bài 26. Kiểm tra tam giác.....	20
Bài 27. Chuyển đổi ngày sang tháng, tuần, ngày	21
Bài 28. Phương trình bậc 2.....	22
Bài 29. Số thuộc đoạn	24
Bài 30. Phép chia	25
Bài 31. Kết quả học tập	25
Bài 32. Số nhỏ thứ 2	26
Bài 33 Tính tổng	27
Bài 34 Xếp thẻ	28
Bài 35 Tính số viên gạch.....	28
Bài 36 Vị trí của Ếch.....	29
Bài 37 Kiểm tra di chuyển.....	30
Bài 38. Mua nước.....	30
Bài 39 Số lượng đồng xu	31
Bài 40 Số tờ tiền tối thiểu.....	32
Bài 41 Số bước leo cầu thang.....	32
Bài 42 Tìm quãng đường ngắn nhất.....	33
Bài 43 Số tiền cần vay.....	34

3. NỘI DUNG CHI TIẾT CÁC BÀI TẬP

Bài 1. Phần nguyên, phần dư

Nội dung: Tính và in ra phần nguyên, phần dư của phép chia 2 số nguyên a, b .

Input: 2 số nguyên a, b với b khác 0 ($-10^{18} \leq a, b \leq 10^{18}$)

Output: Đáp án của bài toán được in trên 1 dòng

Ví dụ:

Input	Output
100 5	20 0
14 6	2 2

- Ý tưởng:** Nhập 2 số nguyên a, b ($b \neq 0$). In ra a/b và $a \% b$. Chú ý $a, b \in [-10^{18}, 10^{18}]$ nên dùng kiểu long long.

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
```

```
    long long a, b;
```

```
    cin >> a >> b;
```

```
    cout << a / b << " " << a % b << endl;
```

```
    return 0;
```

```
}
```

Bài 2. Tính toán giá trị biểu thức

Nội dung: Cho biểu thức $A(x) = x^3 + 3x^2 + x + 1$

Với giá trị của x được nhập từ bàn phím, tính và in ra giá trị của biểu thức trên.

Input: Số nguyên dương x không quá 10^5

Output: Kết quả của biểu thức $A(x)$

Ví dụ:

Input	Output
2	23

- Ý tưởng:** Tính $A(x) = x^3 + 3x^2 + x + 1$ với $x \leq 10^5$. Kết quả có thể vượt quá giới hạn số nguyên 32-bit nên dùng kiểu long long.

```
#include <iostream>
```

```
using namespace std;
```

```

int main() {
    long long x;

    cin >> x;

    long long ans = x * x * x + 3 * x * x + x + 1;

    cout << ans << endl;

    return 0;
}

```

Bài 3. Tính toán giá trị biểu thức 2

Nội dung: Yêu cầu: Cho ba số nguyên dương a , b và c , hãy tính $S = a \cdot (b+c) + b \cdot (a+c)$.

Input: Một dòng ba số nguyên a , b , c ($0 < |a|, |b|, |c| < 10^9$), a , b , c cách nhau một khoảng trắng.

Output: Một dòng ghi giá trị $S = a \cdot (b+c) + b \cdot (a+c)$.

Ví dụ:

Input	Output
1 2 3	13

- **Ý tưởng:** Tính $S = a \cdot (b+c) + b \cdot (a+c)$ với $a, b, c < 10^{10}$. Do a, b, c lớn, giá trị tích có thể đạt $2 \cdot 10^{20}$ (vượt qua giới hạn 64-bit của long long), ta sử dụng kiểu `__int128` trong C++.

```

#include <iostream>
#include <string>
#include <algorithm>
using namespace std;
void print128(__int128 n) {
    if (n == 0) {
        cout << 0 << endl;
        return;
    }
    string s = "";
    while (n > 0) {
        s += (char)(n % 10 + '0');
        n /= 10;
    }
}

```

```

reverse(s.begin(), s.end());

cout << s << endl;
}

int main() {
    long long a, b, c;
    cin >> a >> b >> c;
    __int128 A = a, B = b, C = c;
    __int128 S = A * (B + C) + B * (A + C);
    print128(S);
    return 0;
}

```

Bài 4. Tính tổng, hiệu, tích, thương

Nội dung: Nhập vào 2 số nguyên, in ra tổng, hiệu, tích, thương (lấy độ chính xác với 2 chữ số thập phân).

Input: 2 số nguyên a, b với b khác 0 ($-10^9 \leq a, b \leq 10^9$)

Output: Tổng, hiệu, tích, thương của 2 số

Ví dụ:

Input	Output
10 2	12 8 20 5.00
1000000 1000000	2000000 0 1000000000000 1.00

- **Ý tưởng:** Nhập a, b ($-10^9 \leq a, b \leq 10^9, b \neq 0$). In ra tổng, hiệu, tích (dùng long long), thương (ép kiểu double, làm tròn 2 chữ số thập phân).

```

#include <iostream>
#include <iomanip>
using namespace std;
int main() {
    long long a, b;
    cin >> a >> b;
    cout << a + b << " " << a - b << " " << a * b << " ";
    cout << fixed << setprecision(2) << (double)a / b << endl;
    return 0;
}

```

}

Bài 5. Tính chu vi, diện tích hình tròn**Nội dung:** Nhập bán kính r của hình tròn. Tính chu vi và diện tích hình tròn.**Input:** Bán kính r của hình tròn là một số nguyên ($1 \leq r \leq 10^6$)**Output:** Chu vi và diện tích của hình tròn lấy độ chính xác với 2 chữ số thập phân**Ví dụ:**

Input	Output
10	62.80 314.00

- Ý tưởng:** Chu vi $C = 2 \cdot \pi \cdot r$, Diện tích $S = \pi \cdot r^2$ với $\pi = 3.14$. In kết quả làm tròn 2 chữ số thập phân.

#include <iostream>

#include <iomanip>

using namespace std;

int main() {

double r;

cin >> r;

double PI = 3.14;

double cv = 2 * PI * r;

double dt = PI * r * r;

cout << fixed << setprecision(2) << cv << " " << dt << endl;

return 0;

}

Bài 6. Tính khoảng cách**Nội dung:** Tính khoảng cách Euclid giữa 2 điểm trong hệ tọa độ Oxy.**Input:** Tọa độ của 2 điểm (x_1, y_1) và (x_2, y_2) là các số nguyên ($-10^6 \leq x_i, y_i \leq 10^6$)**Output:** Khoảng cách giữa 2 điểm lấy độ chính xác với 2 chữ số thập phân**Ví dụ:**

Input	Output
1 4 4 8	5.00

- Ý tưởng:** Khoảng cách Euclid giữa $A(x_1, y_1)$ và $B(x_2, y_2)$ là $d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$.

#include <iostream>

```
#include <cmath>

#include <iomanip>

using namespace std;

int main() {

    double x1, y1, x2, y2;

    cin >> x1 >> y1 >> x2 >> y2;

    double d = sqrt((x1 - x2) * (x1 - x2) + (y1 - y2) * (y1 - y2));

    cout << fixed << setprecision(2) << d << endl;

    return 0;

}
```

Bài 7. Chuyển đơn vị đo C và F

Nội dung: Công thức chuyển đơn vị đo nhiệt độ từ C sang F như sau: $F = (C * 9 / 5) + 32$

Viết chương trình C cho phép nhập vào nhiệt độ đo theo độ C là số nguyên dương không quá 10^6 , thực hiện chuyển sang đơn vị đo độ F và in ra màn hình. (Lưu ý luôn lấy 2 chữ số thập phân sau dấu chấm phẩy)

Input: Nhiệt độ C là số nguyên dương không quá 10^6 (ví dụ: 24)

Output: Nhiệt độ theo độ F lấy 2 chữ số thập phân

Ví dụ:

Input	Output
24	75.20

- **Ý tưởng:** Sử dụng công thức $F = (C \cdot 9/5) + 32$.

```
#include <iostream>

#include <iomanip>

using namespace std;

int main() {

    double c;

    cin >> c;

    double f = (c * 9.0 / 5.0) + 32;

    cout << fixed << setprecision(2) << f << endl;

    return 0;

}
```

Bài 8. Tính tổng 1**Nội dung:** $S_n = 1 + 2 + 3 + 4 + \dots + n$ **Input:** Số nguyên không âm n ($0 \leq n \leq 10^8$)**Output:** Kết quả của bài toán**Ví dụ:**

Input	Output
1000000000	500000000500000000

- **Ý tưởng:** $S_n = 1 + 2 + \dots + n = \frac{n(n+1)}{2}$. Áp dụng công thức để tính trong $O(1)$.

#include <iostream>

using namespace std;

int main() {

long long n;

cin >> n;

long long ans = n * (n + 1) / 2;

cout << ans << endl;

return 0;

}

Bài 9. Tính tổng 2**Nội dung:** $S_n = 1^2 + 2^2 + 3^2 + 4^2 + 5^2 + \dots + n^2$ **Input:** Số nguyên không âm n ($0 \leq n \leq 10^5$)**Output:** Kết quả của bài toán**Ví dụ:**

Input	Output
100000	333338333350000

- **Ý tưởng:** $S_n = 1^2 + 2^2 + \dots + n^2 = \frac{n(n+1)(2n+1)}{6}$.

#include <iostream>

using namespace std;

int main() {

long long n;

cin >> n;

```

long long ans = n * (n + 1) * (2 * n + 1) / 6;

cout << ans << endl;

return 0;
}

```

Bài 10. Tính tổng 3

Nội dung: $S_n = 1/(1*2) + 1/(2*3) + 1/(3*4) + \dots + 1/(n*(n+1))$

Input: Số nguyên dương n ($1 \leq n \leq 10^9$)

Output: Kết quả của bài toán lấy độ chính xác 2 chữ số

Ví dụ:

Input	Output
99	0.99

- **Ý tưởng:** $S_n = \frac{1}{1 \cdot 2} + \frac{1}{2 \cdot 3} + \dots + \frac{1}{n(n+1)} = 1 - \frac{1}{n+1} = \frac{n}{n+1}$.

```

#include <iostream>

#include <iomanip>

using namespace std;

int main() {

    double n;

    cin >> n;

    double ans = n / (n + 1);

    cout << fixed << setprecision(2) << ans << endl;

    return 0;

}

```

Bài 11. Tính tổng 4

Nội dung: $S_n = 2 + 4 + 6 + 8 + \dots + 2*n$

Input: Số nguyên dương n ($1 \leq n \leq 10^9$)

Output: Kết quả của bài toán

Ví dụ:

Input	Output
1000000	1000001000000
3	12

- **Ý tưởng:** $S_n = 2 + 4 + 6 + \dots + 2n = 2 \cdot (1 + 2 + \dots + n) = n(n + 1)$.

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
    long long n;
    cin >> n;
    long long ans = n * (n + 1);
    cout << ans << endl;
    return 0;
}
```

Bài 12. Tính tổng 5

Nội dung: $S_n = -1 + 2 - 3 + 4 - 5 + 6 + \dots + (-1)^n \cdot n$

Input: Số nguyên dương n ($1 \leq n \leq 10^{16}$)

Output: Kết quả của bài toán

Ví dụ:

Input	Output
10000000000000000	5000000000000000

- **Ý tưởng:** $S_n = -1 + 2 - 3 + 4 - \dots + (-1)^n n$.

- Nếu n chẵn: $S_n = \frac{n}{2}$.
- Nếu n lẻ: $S_n = \frac{n-1}{2} - n = -\frac{n+1}{2}$.

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
    long long n;
    cin >> n;
    if (n % 2 == 0) cout << n / 2 << endl;
    else cout << -(n + 1) / 2 << endl;
    return 0;
}
```

}

Bài 13. Số chia hết lớn nhất

Nội dung: Cho 2 số nguyên dương a và b . Tìm số chia hết cho b lớn nhất và không vượt quá a . Chú ý không dùng vòng lặp và các hàm có sẵn.

Input: 2 số nguyên dương a, b ($1 \leq b \leq a \leq 10^8$)

Output: Kết quả của bài toán

Ví dụ:

Input	Output
19 5	15
20 5	20

- **Ý tưởng:** Tìm số $\leq a$ lớn nhất chia hết cho b là $(a / b) * b$.

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
```

```
    long long a, b;
```

```
    cin >> a >> b;
```

```
    cout << (a / b) * b << endl;
```

```
    return 0;
```

```
}
```

Bài 14. Số chia hết nhỏ nhất

Nội dung: Cho 2 số nguyên dương a và b . Tìm số chia hết cho b nhỏ nhất và lớn hơn hoặc bằng a . Chú ý không dùng vòng lặp và các hàm có sẵn.

Input: 2 số nguyên dương a, b ($1 \leq b \leq a \leq 10^8$)

Output: Kết quả của bài toán

Ví dụ:

Input	Output
19 5	20
20 5	20
21 5	25

- **Ý tưởng:** Tìm số $\geq a$ nhỏ nhất chia hết cho b là $((a + b - 1) / b) * b$.

```
#include <iostream>

using namespace std;

int main() {
    long long a, b;

    cin >> a >> b;

    cout << ((a + b - 1) / b) * b << endl;

    return 0;
}
```

Bài 15. Kiểm tra số chia hết cho 3 và 5

Nội dung: Kiểm tra xem một số nguyên n có chia hết cho cả 3 và 5 hay không.

Input: Số nguyên n ($-10^{18} \leq n \leq 10^{18}$)

Output: In ra 1 nếu n chia hết cho cả 3 và 5, ngược lại in ra 0

Ví dụ:

Input	Output
30	1
25	0

- **Ý tưởng:** Kiểm tra $n \% 3 == 0 \ \&\& \ n \% 5 == 0$.

```
#include <iostream>

using namespace std;

int main() {
    long long n;

    cin >> n;

    if (n % 3 == 0 && n % 5 == 0) cout << 1 << endl;

    else cout << 0 << endl;

    return 0;
}
```

Bài 16. Kiểm tra năm nhuận

Nội dung: Năm nhuận là năm chia hết cho 400 hoặc (chia hết cho 4 và không chia hết cho 100).

Input: Năm là một số nguyên ($-10^6 \leq n \leq 10^6$)

Output: In ra INVALID nếu n là một số nguyên âm hoặc số 0. Nếu n là năm nhuận, in ra YES, ngược lại in ra NO.

Ví dụ:

Input	Output
2021	NO
2020	YES
-1982	INVALID

- Ý tưởng:

- Nếu $n \leq 0 \rightarrow$ in INVALID.
- Nếu $n \% 400 == 0$ hoặc $(n \% 4 == 0 \text{ và } n \% 100 \neq 0) \rightarrow$ in YES, ngược lại NO.

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
```

```
    long long n;
```

```
    cin >> n;
```

```
    if (n <= 0) {
```

```
        cout << "INVALID" << endl;
```

```
    } else {
```

```
        if ((n % 400 == 0) || (n % 4 == 0 && n % 100 != 0)) cout << "YES" << endl;
```

```
        else cout << "NO" << endl;
```

```
    }
```

```
    return 0;
```

```
}
```



Bài 17. In ra số ngày của tháng

Nội dung: Nhập vào tháng t và năm n . In ra số ngày tương ứng của tháng đó.

Input: 2 số nguyên t, n lần lượt là tháng và năm ($-10^6 \leq t, n \leq 10^6$)

Output: Nếu tháng và năm nhập vào không hợp lệ (tháng, năm không phải số dương, tháng không thuộc $[1, 12]$) in ra INVALID, ngược lại in ra số ngày trong tháng. Chú ý tháng 2 của năm nhuận có 29 ngày.

Ví dụ:

Input	Output
2 2021	28
1 2021	31

14 2020	INVALID
-1 2019	INVALID
2 2020	29

- Ý tưởng:

- Điều kiện không hợp lệ: $t < 1$ hoặc $t > 12$ hoặc $n \leq 0 \rightarrow \text{INVALID}$.
- Tháng 1, 3, 5, 7, 8, 10, 12 có 31 ngày.
- Tháng 4, 6, 9, 11 có 30 ngày.
- Tháng 2 có 29 ngày nếu năm nhuận, 28 ngày nếu năm thường.

```
#include <iostream>
```

```
using namespace std;
```

```
bool isLeap(long long n) {
```

```
    return (n % 400 == 0) || (n % 4 == 0 && n % 100 != 0);
```

```
}
```

```
int main() {
```

```
    long long t, n;
```

```
    cin >> t >> n;
```

```
    if (t < 1 || t > 12 || n <= 0) {
```

```
        cout << "INVALID" << endl;
```

```
    } else {
```

```
        if (t == 1 || t == 3 || t == 5 || t == 7 || t == 8 || t == 10 || t == 12) {
```

```
            cout << 31 << endl;
```

```
        } else if (t == 4 || t == 6 || t == 9 || t == 11) {
```

```
            cout << 30 << endl;
```

```
        } else {
```

```
            if (isLeap(n)) cout << 29 << endl;
```

```
            else cout << 28 << endl;
```

```
        }
```

```
    }
```

```
    return 0;
```

}

Bài 18. Kiểm tra chữ in thường**Nội dung:** Kiểm tra kí tự nhập vào có phải là chữ cái in thường hay không.**Input:** Kí tự cần kiểm tra**Output:** In ra YES nếu kí tự nhập vào là chữ cái in thường, NO trong trường hợp ngược lại**Ví dụ:**

Input	Output
A	NO
a	YES
%	NO

Giải:

#include <iostream>

using namespace std;

int main() {

char c;

cin >> c;

if (c >= 'a' && c <= 'z') cout << "YES" << endl;

else cout << "NO" << endl;

return 0;

}

Bài 19. Kiểm tra in hoa**Nội dung:** Kiểm tra kí tự nhập vào có phải là chữ cái in hoa hay không.**Input:** Kí tự cần kiểm tra**Output:** In ra YES nếu kí tự nhập vào là chữ cái in hoa, NO trong trường hợp ngược lại**Ví dụ:**

Input	Output
A	YES
a	NO
%	NO

Giải:

#include <iostream>

```
using namespace std;

int main() {
    char c;

    cin >> c;

    if (c >= 'A' && c <= 'Z') cout << "YES" << endl;

    else cout << "NO" << endl;

    return 0;
}
```

Bài 20. Kiểm tra chữ cái

Nội dung: Kiểm tra kí tự nhập vào có phải là chữ cái (hoa hoặc thường) hay không.

Input: Kí tự cần kiểm tra

Output: In ra YES nếu kí tự nhập vào là chữ cái, NO trong trường hợp ngược lại

Ví dụ:

Input	Output
A	YES
a	YES
%	NO
1	NO

Giải:

```
#include <iostream>

using namespace std;

int main() {
    char c;

    cin >> c;

    if ((c >= 'a' && c <= 'z') || (c >= 'A' && c <= 'Z')) cout << "YES" << endl;

    else cout << "NO" << endl;

    return 0;
}
```

Bài 21. Kiểm tra chữ số

Nội dung: Kiểm tra kí tự nhập vào có phải là chữ số ('0' đến '9') hay không.

Input: Kí tự cần kiểm tra

Output: In ra YES nếu kí tự nhập vào là chữ số, NO trong trường hợp ngược lại

Ví dụ:

Input	Output
A	NO
a	NO
5	YES

Giải:

```
#include <iostream>

using namespace std;

int main() {
    char c;

    cin >> c;

    if (c >= '0' && c <= '9') cout << "YES" << endl;
    else cout << "NO" << endl;

    return 0;
}
```

Bài 22. Chuyển ký tự hoa thành thường

Nội dung: Chuyển ký tự in hoa thành ký tự in thường tương ứng.

Input: Kí tự cần chuyển

Output: Nếu kí tự nhập vào là chữ in hoa, in ra dạng in thường tương ứng của nó. Trong trường hợp kí tự nhập vào không phải là chữ in hoa thì không thay đổi kí tự ban đầu.

Ví dụ:

Input	Output
A	a
a	a
%	%

Giải:

```
#include <iostream>
```

```
using namespace std;

int main() {
    char c;

    cin >> c;

    if (c >= 'A' && c <= 'Z') c += 32;

    cout << c << endl;

    return 0;
}
```

Bài 23. Chuyển ký tự thường thành ký tự hoa

Nội dung: Chuyển ký tự in thường thành ký tự in hoa tương ứng.

Input: Ký tự cần chuyển

Output: Nếu ký tự nhập vào là chữ in thường, in ra dạng in hoa tương ứng của nó. Trong trường hợp ký tự nhập vào không phải là chữ in thường thì không thay đổi ký tự ban đầu.

Ví dụ:

Input	Output
A	A
a	A
%	%

Giải:

```
#include <iostream>

using namespace std;

int main() {
    char c;

    cin >> c;

    if (c >= 'a' && c <= 'z') c -= 32;

    cout << c << endl;

    return 0;
}
```

Bài 24. Chữ cái kế tiếp

Nội dung: In ra chữ cái kế tiếp của ký tự nhập vào trong bảng chữ cái ở dạng in thường. Ta coi chữ cái kế tiếp của 'z' là 'a'.

Input: Kí tự duy nhất

Output: Nếu kí tự nhập vào là chữ cái, tiến hành in ra chữ cái kế tiếp ở dạng in thường. Nếu không phải chữ cái in ra INVALID.

Ví dụ:

Input	Output
A	b
Z	a
1	INVALID
\$	INVALID

- **Ý tưởng:** Kiểm tra xem c có phải chữ cái hay không. Chuyển c về chữ in thường, nếu là 'z' thì chuyển thành 'a', ngược lại tăng giá trị mã ASCII lên 1. Nếu không phải chữ cái in INVALID.

```
#include <iostream>
```

```
#include <cctype>
```

```
using namespace std;
```

```
int main() {
```

```
    char c;
```

```
    cin >> c;
```

```
    if ((c >= 'a' && c <= 'z') || (c >= 'A' && c <= 'Z')) {
```

```
        char lower_c = tolower(c);
```

```
        if (lower_c == 'z') cout << 'a' << endl;
```

```
        else cout << (char)(lower_c + 1) << endl;
```

```
    } else {
```

```
        cout << "INVALID" << endl;
```

```
    }
```

```
    return 0;
```

```
}
```

Bài 25. Tam giác hợp lệ

Nội dung: Kiểm tra 3 số a, b, c có thể là độ dài 3 cạnh của một tam giác hay không.

Input: a, b, c là độ dài 3 cạnh của tam giác ($-10^6 \leq a, b, c \leq 10^6$)

Output: In ra YES nếu tam giác nhập vào là hợp lệ, ngược lại in NO

Ví dụ:

Input	Output
3 4 5	YES
1 1 5	NO
-1 2 3	NO
0 4 5	NO

- **Ý tưởng:** $a, b, c > 0$ và bất đẳng thức tam giác: $a + b > c, a + c > b, b + c > a$.

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
```

```
    long long a, b, c;
```

```
    cin >> a >> b >> c;
```

```
    if (a > 0 && b > 0 && c > 0 && (a + b > c) && (a + c > b) && (b + c > a)) {
```

```
        cout << "YES" << endl;
```

```
    } else {
```

```
        cout << "NO" << endl;
```

```
    }
```

```
    return 0;
```

```
}
```

Bài 26. Kiểm tra tam giác

Nội dung: Cho 3 cạnh a, b, c của tam giác. Phân loại loại tam giác.

Input: a, b, c là độ dài 3 cạnh của tam giác ($-10^6 \leq a, b, c \leq 10^6$)

Output: In ra INVALID nếu tam giác không hợp lệ.

In ra 1 nếu là tam giác đều.

In ra 2 nếu là tam giác cân.

In ra 3 nếu là tam giác vuông.

In ra 4 nếu là tam giác thường.

Ví dụ:

Input	Output
3 4 5	3
3 3 3	1

1 1 8	INVALID
4 4 6	2

- **Ý tưởng:**

1. Kiểm tra tính hợp lệ của tam giác → Không hợp lệ in INVALID.
2. $a = b = c \rightarrow$ Tam giác đều (in 1).
3. $a = b$ hoặc $b = c$ hoặc $a = c \rightarrow$ Tam giác cân (in 2).
4. Định lý Pitago $\rightarrow$ Tam giác vuông (in 3).
5. Ngược lại $\rightarrow$ Tam giác thường (in 4).

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
```

```
    long long a, b, c;
```

```
    cin >> a >> b >> c;
```

```
    if (a <= 0 || b <= 0 || c <= 0 || (a + b <= c) || (a + c <= b) || (b + c <= a)) {
```

```
        cout << "INVALID" << endl;
```

```
    } else if (a == b && b == c) {
```

```
        cout << 1 << endl;
```

```
    } else if (a == b || b == c || a == c) {
```

```
        cout << 2 << endl;
```

```
    } else if (a * a + b * b == c * c || a * a + c * c == b * b || b * b + c * c == a * a) {
```

```
        cout << 3 << endl;
```

```
    } else {
```

```
        cout << 4 << endl;
```

```
    }
```

```
    return 0;
```

```
}
```

Bài 27. Chuyển đổi ngày sang tháng, tuần, ngày

Nội dung: Viết chương trình C cho phép nhập vào số ngày, thực hiện chuyển số ngày sang năm, tuần, ngày (Bỏ qua trường hợp năm nhuận).

Input: Số nguyên n không âm ($0 \leq n \leq 10^6$)

Output: Chuyển số ngày đã cho sang số năm, số tuần, số ngày

Ví dụ:

Input	Output
373	1 1 1

- **Ý tưởng:** 1 năm = 365 ngày, 1 tuần = 7 ngày.
 - Số năm = $n/365$, số ngày còn lại $n = n\%365$.
 - Số tuần = $n/7$, số ngày dư = $n\%7$.

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
    int n;
    cin >> n;
    int nam = n / 365;
    n %= 365;
    int tuan = n / 7;
    int ngay = n % 7;
    cout << nam << " " << tuan << " " << ngay << endl;
    return 0;
}
```



Bài 28. Phương trình bậc 2

Nội dung: Phương trình bậc 2 là phương trình dạng $ax^2 + bx + c = 0$.

Viết chương trình C cho phép nhập vào a, b, c và thực hiện giải phương trình bậc 2. Nếu vô nghiệm thì in ra dòng NO, nếu vô số nghiệm thì in ra INF. Nếu có nghiệm thì in các nghiệm (luôn lấy 2 chữ số thập phân sau dấu chấm phẩy) cách nhau một khoảng trắng.

Input: Một dòng gồm 3 số a, b, c

Output: Nghiệm của phương trình (hoặc NO / INF)

Ví dụ:

Input	Output
8 -4 -2	0.81 -0.31

- **Ý tưởng:** Giải phương trình $ax^2 + bx + c = 0$.

- Nếu $a = 0$: Trở thành phương trình bậc nhất $bx + c = 0$.
- Nếu $a \neq 0$: Tính $\Delta = b^2 - 4ac$.
 - $\Delta < 0 \rightarrow$ in NO.
 - $\Delta = 0 \rightarrow$ Nghiệm kép $x = -b/(2a)$.
 - $\Delta > 0 \rightarrow$ 2 nghiệm phân biệt $x_1 = (-b + \sqrt{\Delta})/(2a)$, $x_2 = (-b - \sqrt{\Delta})/(2a)$.

```
#include <iostream>

#include <cmath>

#include <iomanip>

using namespace std;

int main() {
    double a, b, c;
    cin >> a >> b >> c;
    if (a == 0) {
        if (b == 0) {
            if (c == 0) cout << "INF" << endl;
            else cout << "NO" << endl;
        } else {
            cout << fixed << setprecision(2) << -c / b << endl;
        }
    } else {
        double delta = b * b - 4 * a * c;
        if (delta < 0) {
            cout << "NO" << endl;
        } else if (delta == 0) {
            cout << fixed << setprecision(2) << -b / (2 * a) << endl;
        } else {
            double x1 = (-b + sqrt(delta)) / (2 * a);
            double x2 = (-b - sqrt(delta)) / (2 * a);
        }
    }
}
```

```

        cout << fixed << setprecision(2) << x1 << " " << x2 << endl;
    }
}

return 0;
}

```

Bài 29. Số thuộc đoạn

Nội dung: Yêu cầu: Cho một đoạn đại số $[a, b]$. Tính số lượng số nguyên trong đoạn $[a, b]$ đó.

Input: Một dòng ghi 2 số thực a, b .

Output: Số lượng các số nguyên trong đoạn $[a, b]$.

Ví dụ:

Input	Output
1.1 5.2	4

- Ý tưởng:** Cho hai số thực a, b . Số nguyên nằm trong khoảng $[a, b]$ bắt đầu từ $[a]$ đến $[b]$. Số lượng số nguyên là $[b] - [a] + 1$ (nếu $[a] \leq [b]$).

```

#include <iostream>
#include <cmath>
#include <algorithm>
using namespace std;
int main() {
    double a, b;
    cin >> a >> b;
    if (a > b) swap(a, b);
    long long left = ceil(a);
    long long right = floor(b);
    if (left <= right) cout << right - left + 1 << endl;
    else cout << 0 << endl;
    return 0;
}

```

Bài 30. Phép chia

Nội dung: Yêu cầu: Cho 3 số nguyên 64 bit a, b, c. In ra dấu / nếu $a/b=c$ hoặc $b/c=a$ hoặc $c/a=b$ và in ra NOSOL nếu không thỏa mãn.

Input: Một dòng gồm 3 số nguyên a, b, c.

Output: Ghi ra / nếu thỏa mãn chia hết hoặc in ra NOSOL nếu không thỏa mãn.

Ví dụ:

Input	Output
3 1 3	/
3 4 5	NOSOL

- **Ý tưởng:** Kiểm tra điều kiện chia hết thích hợp ($a/b = c$, $b/c = a$, $c/a = b$). In / nếu thỏa mãn, ngược lại in NOSOL.

```
#include <iostream>

using namespace std;

int main() {
    long long a, b, c;
    cin >> a >> b >> c;
    bool check = false;
    if (b != 0 && a % b == 0 && a / b == c) check = true;
    if (c != 0 && b % c == 0 && b / c == a) check = true;
    if (a != 0 && c % a == 0 && c / a == b) check = true;
    if (check) cout << "/" << endl;
    else cout << "NOSOL" << endl;
    return 0;
}
```

Bài 31. Kết quả học tập

Nội dung: Yêu cầu: Cho biết điểm kiểm tra Tin học của 1 em học sinh (2 con điểm hệ số 1, 1 con điểm hệ số 2, 1 con điểm hệ số 3). In ra Kết quả học tập môn Tin học của em đó. Nếu điểm tổng kết ≥ 8 đạt kết quả Giỏi, < 8 và ≥ 6.5 đạt kết quả Khá, < 6.5 và ≥ 5 đạt kết quả Trung Bình, < 5 đạt kết quả Yếu.

Input: Một dòng chứa 4 số điểm của học sinh.

Output: Kết quả học tập môn Tin học của em học sinh ở dạng in hoa không dấu.

Ví dụ:

Input	Output
-------	--------

9 8 7 8.5	GIOI
5 7 6.5 5	TRUNG BINH

- **Ý tưởng:** Tính điểm tổng kết $T = \frac{d_1 + d_2 + 2 \cdot d_3 + 3 \cdot d_4}{7}$.
 - $T \geq 8.0 \rightarrow \text{GIOI}$
 - $6.5 \leq T < 8.0 \rightarrow \text{KHA}$
 - $5.0 \leq T < 6.5 \rightarrow \text{TRUNG BINH}$
 - $T < 5.0 \rightarrow \text{YEU}$

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
```

```
    double d1, d2, d3, d4;
```

```
    cin >> d1 >> d2 >> d3 >> d4;
```

```
    double tk = (d1 + d2 + d3 * 2 + d4 * 3) / 7.0;
```

```
    if (tk >= 8.0) cout << "GIOI" << endl;
```

```
    else if (tk >= 6.5) cout << "KHA" << endl;
```

```
    else if (tk >= 5.0) cout << "TRUNG BINH" << endl;
```

```
    else cout << "YEU" << endl;
```

```
    return 0;
```

```
}
```

Bài 32. Số nhỏ thứ 2

Nội dung: Yêu cầu: Cho 5 số nguyên a, b, c, d, e 64 bit đôi một khác nhau. In ra số nhỏ thứ nhì.

Input: Một dòng gồm 5 số nguyên a, b, c, d, e.

Output: In ra số nhỏ thứ nhì.

Ví dụ:

Input	Output
1 2 3 4 5	2

- **Ý tưởng:** Nhập 5 số nguyên 64-bit vào mảng, sắp xếp mảng tăng dần bằng `std::sort` và in phần tử ở vị trí chỉ số 1.

```
#include <iostream>
```

```
#include <algorithm>
```

```
using namespace std;

int main() {
    long long a[5];

    for (int i = 0; i < 5; i++) {
        cin >> a[i];
    }

    sort(a, a + 5);

    cout << a[1] << endl;

    return 0;
}
```

Bài 33 Tính tổng

Cho biết diện tích của ba mặt có chung đỉnh của hình hộp chữ nhật, tính tổng độ dài 12 cạnh của hình hộp chữ nhật đó.

Input: 3 số nguyên dương không vượt quá 10^4 là diện tích của ba mặt có chung đỉnh.

Output: Tổng của tất cả các cạnh của hình hộp chữ nhật.

Ví dụ:

Input	Output
4 6 6	28

• **Ý tưởng:**

- Gọi độ dài 3 cạnh chung đỉnh của hình hộp chữ nhật lần lượt là a, b, c ($a, b, c > 0$).
- Diện tích ba mặt chung đỉnh lần lượt là:
 - $S1 = a * b$
 - $S2 = b * c$
 - $S3 = c * a$
- Nhân cả 3 phương trình lại ta có: $S1 * S2 * S3 = (a * b * c)^2$
 $\Rightarrow a * b * c = \sqrt{S1 * S2 * S3}$
- Suy ra độ dài từng cạnh:
 - $a = (a * b * c) / S2 = \sqrt{S1 * S2 * S3} / S2$
 - $b = (a * b * c) / S3 = \sqrt{S1 * S2 * S3} / S3$
 - $c = (a * b * c) / S1 = \sqrt{S1 * S2 * S3} / S1$
- Tổng độ dài 12 cạnh của hình hộp chữ nhật là: $4 * (a + b + c)$.

Code:

```
#include <iostream>
#include <cmath>
using namespace std;
int main() {
```

```

long long s1, s2, s3;
if (cin >> s1 >> s2 >> s3) {
    long long abc = sqrt(s1 * s2 * s3);
    long long a = abc / s2;
    long long b = abc / s3;
    long long c = abc / s1;
    long long total_edges = 4 * (a + b + c);
    cout << total_edges << endl;
}
return 0;
}

```

Bài 34 Xếp thẻ

Cho bảng hình chữ nhật kích thước $M \times N$ ô vuông. Đặt các thanh Domino kích thước 2×1 vào bảng sao cho phủ kín tối đa các ô vuông mà không đè lên nhau. Hãy tính số thanh Domino tối đa có thể đặt.

Input: Hai số nguyên M và N ($1 \leq M, N \leq 16$).

Output: Số thanh Domino tối đa có thể đặt.

- **Ý tưởng:**

1. Diện tích tổng cộng của bảng hình chữ nhật $M \times N$ là $M * N$ ô vuông đơn vị.
2. Mỗi thanh domino kích thước 2×1 chiếm đúng 2 ô vuông đơn vị.
3. Vì ta luôn có thể xếp các domino sao cho phủ tối đa diện tích (chỉ bỏ trống nhiều nhất 1 ô nếu $M * N$ lẻ), số lượng domino tối đa có thể đặt được chính là phép chia nguyên: $(M * N) / 2$.

Code:

```

#include <iostream>
using namespace std;
int main() {
    int m, n;
    if (cin >> m >> n) {
        cout << (m * n) / 2 << endl;
    }
    return 0;
}

```

Bài 35 Tính số viên gạch

Quảng trường Nhà hát có kích thước $n \times m$ mét. Cần lát đường bằng các viên gạch vuông kích thước $a \times a$ mét. Tính số lượng viên gạch tối thiểu cần dùng để phủ kín Quảng trường.

Input: Ba số nguyên dương n, m, a ($1 \leq n, m, a \leq 10^9$).

Output: Số lượng viên gạch tối thiểu cần dùng.

- **Ý tưởng:**

1. Để phủ kín chiều dài n bằng các viên đá độ dài a , số viên đá tối thiểu cần thiết là $\text{ceil}(n / a) = (n + a - 1) / a$.
2. Để phủ kín chiều rộng m bằng các viên đá độ dài a , số viên đá tối thiểu cần thiết là $\text{ceil}(m / a) = (m + a - 1) / a$.
3. Vì các viên đá được xếp thành lưới **прямоугольник**, tổng số viên đá cần dùng là tích của hai số trên:

$$\text{Total} = ((n + a - 1) / a) * ((m + a - 1) / a).$$

4. Chú ý: Vì n, m, a có thể lên đến 10^9 , kết quả có thể lên tới 10^{18} nên phải dùng kiểu dữ liệu 64-bit (long long trong C++).

Code:

```
#include <iostream>

using namespace std;

int main() {
    long long n, m, a;
    if (cin >> n >> m >> a) {
        long long rows = (n + a - 1) / a;
        long long cols = (m + a - 1) / a;
        cout << rows * cols << endl;
    }
    return 0;
}
```

Bài 36 Vị trí của Ếch

Một chú ếch xuất phát từ vị trí 0 trên trục số. Ở bước thứ nhất, ếch nhảy a đơn vị về bên phải. Ở bước thứ hai, ếch nhảy b đơn vị về bên trái, và cứ tiếp tục luân phiên như vậy. Xác định vị trí của ếch sau k bước nhảy.

Input: Ba số nguyên a, b, k ($1 \leq a, b, k \leq 10^9$).

Output: Tọa độ của ếch sau k bước.

- **Ý tưởng:**

1. Tổng số bước nhảy là k .
2. Các bước nhảy lẻ (1, 3, 5, ...) là nhảy sang phải thêm a đơn vị. Số lần nhảy sang phải là $(k + 1) / 2$.
3. Các bước nhảy chẵn (2, 4, 6, ...) là nhảy sang trái giảm b đơn vị. Số lần nhảy sang trái là $k / 2$.
4. Vị trí cuối cùng của con ếch = $((k + 1) / 2) * a - (k / 2) * b$.
5. Dùng kiểu dữ liệu long long để tránh tràn số khi a, b, k tới 10^9 .

Code:

```
#include <iostream>

using namespace std;

int main() {
    long long a, b, k;
    if (cin >> a >> b >> k) {
        long long right_steps = (k + 1) / 2;
        long long left_steps = k / 2;
        long long pos = right_steps * a - left_steps * b;
        cout << pos << endl;
    }
    return 0;
}
```

Bài 37 Kiểm tra di chuyển

Drazil bắt đầu di chuyển từ vị trí (0, 0) đến vị trí (a, b). Mỗi bước di chuyển Drazil đi được đúng 1 đơn vị theo hướng Bắc, Nam, Đông hoặc Tây. Cho biết Drazil đã đi đúng s bước, hãy kiểm tra xem Drazil có thể đến được điểm (a, b) hay không.

Input: Ba số nguyên a, b, s ($-10^9 \leq a, b \leq 10^9$; $1 \leq s \leq 2 * 10^9$).

Output: In "YES" nếu có thể đến đúng vị trí (a, b) sau s bước, ngược lại in "NO".

- **Ý tưởng:**

1. Khoảng cách ngắn nhất (khoảng cách Manhattan) từ (0, 0) đến (a, b) là $\text{min_steps} = |a| + |b|$.
2. Điều kiện để có thể tới đúng điểm (a, b) trong chính xác s bước:
 - Số bước s phải lớn hơn hoặc bằng khoảng cách tối thiểu: $s \geq \text{min_steps}$.
 - Nếu $s > \text{min_steps}$, số bước thừa ($s - \text{min_steps}$) phải là số chẵn (vì mỗi lần đi lệch đường rồi quay lại tốn thêm bội số của 2 bước).
3. Kết luận:
 - Nếu $(s \geq \text{min_steps})$ và $((s - \text{min_steps}) \% 2 == 0) \Rightarrow$ In "YES".
 - Ngược lại $\Rightarrow$ In "NO".

Code:

```
#include <iostream>
#include <cmath>

using namespace std;

int main() {
    long long a, b, s;
    if (cin >> a >> b >> s) {
        long long min_steps = abs(a) + abs(b);
        if (s >= min_steps && (s - min_steps) % 2 == 0) {
            cout << "YES" << endl;
        } else {
            cout << "NO" << endl;
        }
    }
    return 0;
}
```

Bài 38. Mua nước

Cửa hàng bán nước có hai loại chai: loại 1 lít giá a đô la và loại 2 lít giá b đô la. Cần mua đúng n lít nước. Hãy tìm số tiền tối thiểu để mua đủ n lít nước.

Input: Ba số nguyên n, a, b ($1 \leq n \leq 10^{12}$; $1 \leq a, b \leq 1000$).

Output: Số tiền tối thiểu cần trả.

- **Ý tưởng:**

1. Chi phí mua 2 lít nước bằng 2 chai loại 1L là $2 * a$.
2. Chi phí mua 2 lít nước bằng 1 chai loại 2L là b.
3. So sánh $2 * a$ và b:
 - Nếu $2 * a \leq b$: Mua bằng chai 1L rẻ hơn hoặc bằng chai 2L. Khi đó nên mua toàn bộ n lít bằng chai 1L

=> Chi phí = $n * a$.

- Nếu $2 * a > b$: Mua bằng chai 2L rẻ hơn. Ta mua tối đa số chai 2L có thể, tức là $(n / 2)$ chai 2L. Nếu n lẻ ($n \% 2 == 1$), mua thêm 1 chai 1L cho 1 lít còn lại => Chi phí = $(n / 2) * b + (n \% 2) * a$.

4. Dùng kiểu dữ liệu long long cho n và kết quả tính toán.

Code:

```
#include <iostream>

using namespace std;

int main() {
    long long n, a, b;
    if (cin >> n >> a >> b) {
        if (2 * a <= b) {
            cout << n * a << endl;
        } else {
            cout << (n / 2) * b + (n % 2) * a << endl;
        }
    }
    return 0;
}
```

Bài 39 Số lượng đồng xu

Bạn có vô số đồng xu có giá trị từ 1 đến n . Hãy tìm số lượng đồng xu ít nhất cần dùng để tạo thành tổng giá trị đúng bằng S .

Input: Hai số nguyên n và S ($1 \leq n \leq 10^5$, $1 \leq S \leq 10^9$).

Output: Số lượng đồng xu tối thiểu.

- **Ý tưởng:**

1. Để dùng số lượng xu ít nhất, thuật toán tham ăn là chọn càng nhiều đồng xu mệnh giá lớn nhất (mệnh giá n) càng tốt.
2. Số đồng xu mệnh giá n có thể chọn là S / n .
3. Nếu S chia hết cho n ($S \% n == 0$), tổng số xu là S / n .
4. Nếu S không chia hết cho n , phần dư $S \% n$ nhỏ hơn n , ta chỉ cần chọn thêm đúng 1 đồng xu có giá trị bằng $S \% n$. Do đó tổng số xu là $(S / n) + 1$, hay viết gọn dưới dạng phép chia trần (ceil): $(S + n - 1) / n$.

Code:

```
#include <iostream>

using namespace std;

int main() {
    long long n, S;
    if (cin >> n >> S) {
        long long count = (S + n - 1) / n;
        cout << count << endl;
    }
    return 0;
}
```

Bài 40 Số tờ tiền tối thiểu

Ngân hàng cấp các tờ tiền có mệnh giá 1, 5, 10, 20, 100 đô la. Một người muốn rút n đô la sao cho số lượng tờ tiền nhận được là ít nhất. Tính số lượng tờ tiền tối thiểu đó.

Input: Số nguyên n ($1 \leq n \leq 10^9$).

Output: Số tờ tiền ít nhất.

- **Ý tưởng:**

1. Áp dụng thuật toán Tham ăn (Greedy). Xét các mệnh giá tờ tiền theo thứ tự giảm dần: 100, 20, 10, 5, 1.
2. Tại mỗi mệnh giá d :
 - Số tờ tiền nhận được $= n / d$.
 - Số tiền còn lại chưa rút $= n \% d$.
3. Cộng dồn số tờ tiền ở từng mệnh giá cho đến khi rút hết toàn bộ n đô la.

Code:

```
#include <iostream>

using namespace std;

int main() {
    long long n;
    if (cin >> n) {
        long long count = 0;
        int bills[] = {100, 20, 10, 5, 1};
        for (int b : bills) {
            count += n / b;
            n %= b;
        }
        cout << count << endl;
    }
    return 0;
}
```

Bài 41 Số bước leo cầu thang

Dreamoon muốn leo một cầu thang gồm n bậc. Mỗi lần bước, anh ta có thể bước 1 bậc hoặc 2 bậc. Dreamoon muốn tổng số bước đi phải là một bội số của m . Hãy tìm số bước tối thiểu thỏa mãn điều kiện đó.

Input: Hai số nguyên n và m ($1 \leq n \leq 10000$, $2 \leq m \leq 10$).

Output: Số bước tối thiểu là bội số của m . Nếu không có cách nào, in -1.

- **Ý tưởng:**

1. Nếu $n < m$: Số bước nhảy tối thiểu để leo hết n bậc là $\text{ceil}(n / 2) > 0$. Vì max số bước di chuyển là $n < m$, không thể tìm được số bước nhảy nào là bội của $m \Rightarrow$ In -1.
2. Khi $n \geq m$:
 - Số bước di chuyển nhỏ nhất có thể là $\text{min_moves} = (n + 1) / 2$ (khi nhảy nhiều bước 2 nhất có thể).
 - Số bước di chuyển lớn nhất có thể là $\text{max_moves} = n$ (khi chỉ nhảy toàn bước 1).
3. Ta cần tìm số nguyên $\text{ans} \geq \text{min_moves}$ sao cho ans chia hết cho m và ans nhỏ nhất.
4. Bội số nhỏ nhất của m lớn hơn hoặc bằng min_moves được tính công thức (không dùng vòng lặp):

$$\text{ans} = ((\text{min_moves} + m - 1) / m) * m.$$
5. Vì $n \geq m$, ans luôn $\leq n$, thỏa mãn yêu cầu bài toán.

Code:

```
#include <iostream>

using namespace std;

int main() {
    int n, m;
    if (cin >> n >> m) {
        if (n < m) {
            cout << -1 << endl;
        } else {
            int min_moves = (n + 1) / 2;
            int ans = ((min_moves + m - 1) / m) * m;
            cout << ans << endl;
        }
    }
    return 0;
}
```

Bài 42 Tìm quãng đường ngắn nhất

Patrick muốn đến thăm hai cửa hàng xuất phát từ nhà của mình và sau đó trở về nhà. Patrick có thể đi lại qua cùng một cửa hàng hoặc con đường nhiều lần. Tìm khoảng cách tối thiểu Patrick phải đi bộ.

Input: Dòng đầu tiên chứa ba số nguyên d_1, d_2, d_3 ($1 \leq d_1, d_2, d_3 \leq 10^8$) lần lượt là độ dài đường nối từ nhà đến cửa hàng 1, từ nhà đến cửa hàng 2, và đường nối giữa hai cửa hàng.

Output: In khoảng cách tối thiểu Patrick sẽ phải đi bộ.

Ví dụ:

Input	Output
10 20 30	60

- Ý tưởng:**

- Có 4 lộ trình khả thi để Patrick đi từ nhà qua cả 2 cửa hàng rồi quay về nhà:
 - Lộ trình 1: Nhà -> Cửa hàng 1 -> Cửa hàng 2 -> Nhà => Quãng đường = $d_1 + d_3 + d_2$
 - Lộ trình 2: Nhà -> Cửa hàng 1 -> Nhà -> Cửa hàng 2 -> Nhà => Quãng đường = $2*d_1 + 2*d_2$
 - Lộ trình 3: Nhà -> Cửa hàng 1 -> Cửa hàng 2 -> Cửa hàng 1 -> Nhà => Quãng đường = $2*d_1 + 2*d_3$
 - Lộ trình 4: Nhà -> Cửa hàng 2 -> Cửa hàng 1 -> Cửa hàng 2 -> Nhà => Quãng đường = $2*d_2 + 2*d_3$
- Kết quả cần tìm là giá trị nhỏ nhất (min) trong 4 lộ trình trên.
- Dùng kiểu dữ liệu long long để tránh tràn số.

Code:

```
#include <iostream>
#include <algorithm>

using namespace std;

int main() {
    long long d1, d2, d3;
```

```

if (cin >> d1 >> d2 >> d3) {
    long long path1 = d1 + d2 + d3;
    long long path2 = 2 * (d1 + d2);
    long long path3 = 2 * (d1 + d3);
    long long path4 = 2 * (d2 + d3);
    long long ans = min({path1, path2, path3, path4});
    cout << ans << endl;
}
return 0;
}

```

Bài 43 Số tiền cần vay

Một người lính muốn mua w quả chuối. Anh ta phải trả k đô la cho quả chuối đầu tiên, $2k$ đô la cho quả thứ hai, và cứ thế (trả $i * k$ đô la cho quả chuối thứ i). Anh ta đang có n đô la. Hỏi anh ta phải vay bao nhiêu đô la từ người bạn lính của mình để mua đủ chuối?

Input: Dòng đầu tiên chứa ba số nguyên dương k, n, w ($1 \leq k, w \leq 1000, 0 \leq n \leq 10^9$).

Output: Số tiền anh ta cần phải vay. Nếu không cần vay thêm tiền, in ra 0.

- **Ý tưởng:**

1. Tổng số tiền mua w quả chuối là:

$$\text{Total} = k + 2k + 3k + \dots + w \cdot k = k * (1 + 2 + \dots + w) = k * w * (w + 1) / 2.$$

2. Số tiền người lính cần vay thêm là: $\text{borrow} = \text{Total} - n$.

3. Nếu $\text{borrow} > 0$, in ra borrow .

Nếu $\text{borrow} \leq 0$, người lính đã có đủ tiền, in ra 0 (dùng $\max(0, \text{borrow})$).

Code:

```

#include <iostream>
#include <algorithm>

using namespace std;

int main() {
    long long k, n, w;
    if (cin >> k >> n >> w) {
        long long total = k * w * (w + 1) / 2;
        long long borrow = total - n;
        if (borrow < 0) borrow = 0;
        cout << borrow << endl;
    }
    return 0;
}

```